A Dual-Encoding-based Genetic Algorithm for Multi-Objective Multi-UAV Scheduling in Firefighting Scenarios

Meng Gao, Xiao-Fang Liu*, Zhi-Hui Zhan, Jun Zhang
College of Artificial Intelligence
Nankai University
Tianjin, China
liuxiaofang@nankai.edu.cn

2025 IEEE Congress on Evolutionary Computation (CEC) | 979-8-3315-3431-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/CEC65147.2025.11043008

***Abstract*—Unmanned aerial vehicles (UAVs) are increasingly used for firefighting due to security. Multiple UAVs depart from a site and cooperate to execute tasks with time-varying demands in different locations. To better execute firefighting tasks, the reasonable scheduling of UAVs is crucial. Although multiple methods have been developed to solve the multi-UAV scheduling problem, they mainly focus on instances with sufficient UAV resources. They face challenges on instances with limited resources and multiple objectives in terms of solution diversity and convergence, especially when the number of tasks is larger than that of UAVs. This paper models the problem as a bi-objective optimization problem, aiming to minimize two important objectives: the makespan and the total travel distance of UAVs. A dual-encoding-based genetic algorithm (DEGA) is developed to solve the problem. In DEGA, a dual-encoding scheme is adopted for solution representation, in which the execution order of all tasks is represented as a sequence and the task assignment for UAVs is represented as a binary matrix. Correspondingly, solutions can be constructed in two steps: task sequence generation first and then task assignment. Crossover and mutation operations are specifically designed to explore the search space for enhancing solution diversity. In addition, a local search is employed to improve solution quality. Experimental results on instances with various scales demonstrate that DEGA outperforms state-of-the-art algorithms on most instances in terms of solution optimality and diversity.**

***Index Terms*—Genetic algorithm, multi-objective optimization, multi-UAV scheduling, firefighting, multi-robot systems.**

I. Introduction

Unmanned aerial vehicles (UAVs) are widely used in real-world applications, such as monitoring [1] and goods delivery [2]. In firefighting scenarios, multiple UAVs equipped with fire-extinguishing devices form a multi-agent system to eliminate bushfires. Each UAV visits multiple task positions to extinguish fires as quickly as possible [3]. Multi-UAV systems provide higher stability and efficiency compared to single UAV operations [4].

To reduce damage and costs in firefighting tasks, makespan and traveling distances are two important conflicting objectives. The scheduling problem can be formulated as a multi-objective optimization problem [5]. As fires spread over time, the demand at each location increases, making scheduling more challenging. Several evolutionary computation methods have been developed to solve this problem, including genetic algorithms [6] and ant colony optimization [7].

Existing methods like multi-strategy genetic algorithm (MSGA) [8] and adaptive coordination ant colony optimization (ACACO) [9] perform well with sufficient UAV resources but face challenges with limited resources. The problems become more difficult when the number of tasks exceeds the number of UAVs, as constraints are easily violated when determining task assignments and execution orders.

This paper proposes a dual-encoding-based genetic algorithm (DEGA) to address the multi-objective multi-UAV scheduling problem. DEGA adopts a dual-encoding scheme where the execution order of tasks is represented as a sequence and task assignment is represented as a binary matrix. Solutions are generated by first planning the global task sequence and then assigning tasks to UAVs, ensuring that tasks assigned to multiple UAVs follow the same order to meet constraints. Special crossover and mutation operators with repair mechanisms enhance solution diversity, while local search improves convergence.

Experiments on 28 instances with different scales of UAVs and tasks demonstrate that DEGA significantly outperforms state-of-the-art algorithms in terms of solution optimality and diversity. The remainder of the paper presents the problem formulation, details of the proposed algorithm, experimental results, and conclusions.

II. Background

In bushfire elimination problems, multiple fire points (tasks) are distributed across different locations, each with time-varying demands. Multiple UAVs depart from a base station to visit these locations and extinguish fires.

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62476141, in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2025-00555463), and in part by the Tianjin Top Scientist Studio Project under Grant 24JRRCRC00030, and in part by the Tianjin Belt and Road Joint Laboratory under Grant 24PTLYHZ00250. (*Corresponding author: Xiao-Fang Liu)*

We denote the base station as t_0, the task set as $T = \{t_j | 1 \le j \le N\}$, and the UAV set as $U = \{u_i | 1 \le i \le M\}$, where N is the number of tasks and M is the number of UAVs.

As fires grow with time, the demand at task t_j increases according to $q_j(t) = q_j(0) + \gamma_j t$, where t is the time, $q_j(0)$ is the initial demand, and γ_j is the growth rate. Each UAV u_i has a fire elimination capability μ_i per unit time.

A scheduling solution X determines the task execution sequence for each UAV. When UAVs are assigned to a task t_j, their combined capacity must fulfill the task's demand. The completion time e_j of task t_j is determined by:

$$e_j = a_j + \frac{q_j(a_j)}{\sum_{i \in U_j} \mu_i} \tag{1}$$

where a_j is the arrival time of the first UAV and U_j is the set of assigned UAVs. The problem requires each task to be executed by at least one UAV, synchronized execution when multiple UAVs serve the same task, and each UAV can execute a specific task at most once.

Two optimization objectives are considered: minimizing the makespan (maximum completion time) calculated as $f_1(X) = \max_{1 \le j \le N} e_j$, and minimizing the total traveling distance of all UAVs calculated as $f_2(X) = \sum_{i=1}^{M} d_i$, where d_i is the distance traveled by UAV u_i. This results in a bi-objective optimization problem: $\min F(X) = \{f_1(X), f_2(X)\}$, requiring algorithms to find the Pareto front representing the best trade-offs between these conflicting objectives.

III. Method

The proposed DEGA employs a dual-encoding scheme for solution representation. Solutions are generated in two steps: task sequence construction first and then task assignment. Crossover and mutation operators are used to enhance solution diversity. A local search is performed to improve convergence. The details of each component are introduced in the following. The overall flowchart of DEGA is illustrated in Fig. 1.

A. Solution Representation

Solutions are represented using a dual-encoding scheme, which represents the task assignment as a binary matrix and the visiting path of each UAV as a task sequence.

1) Task Sequence: The execution order of all tasks is represented as a task permutation $\mathbf{R} = [r_1, r_2, \ldots, r_N]$, where N is the total number of tasks and each r_i is ith task for execution. The tasks assigned to a UAV will be executed according to their orders in the task permutation.

2) Task Assignment: Task assignment for UAVs is represented as a binary matrix $\mathbf{X} = [x_{ij}]$ with a size of $M \times N$. Each element $x_{ij} \in \{0, 1\}$ indicates whether task t_j is assigned to UAV u_i, where $x_{ij} = 1$ means task t_j is assigned to UAV u_i, and $x_{ij} = 0$ otherwise. This assignment matrix defines which UAVs are responsible for which fire points in the bushfire elimination scenario.

When multiple UAVs are assigned to the same task, they must execute it according to the same order based on a global task sequence. This ensures synchronized execution and maintains consistency across all UAVs, satisfying the constraint that tasks assigned to multiple UAVs follow the same execution order while optimizing for both makespan and total traveling distance.

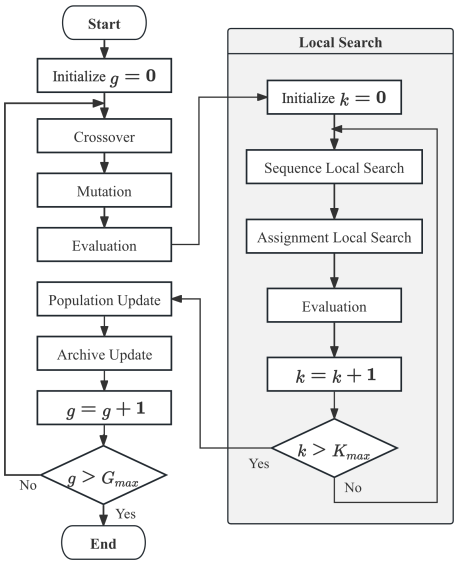

Fig. 1: The flowchart of the proposed DEGA.

B. Genetic Operators

Specific genetic operators are designed based on the dual-encoding scheme to enhance solution diversity.

1) Crossover: For the global task sequence $\mathbf{R}$ of a solution, a modified order crossover is applied. Given two parent sequences $\mathbf{R}_1$ and $\mathbf{R}_2$, two crossover points are randomly selected first, denoted as c_1 and c_2 ($1 \le c_1 < c_2 \le N$). Denote a new task sequence as $\mathbf{R}_{\text{offspring}}$. The subsequence between c_1 and c_2 in $\mathbf{R}_1$ is selected to add in $\mathbf{R}_{\text{offspring}}$. The unvisited tasks in the offspring $\mathbf{R}_{\text{offspring}}$ are inserted in the same order as that in $\mathbf{R}_2$.

For the task assignment matrix $\mathbf{X}$, arithmetic operations are adopted to combine parent matrices. Given two parent matrices $\mathbf{X}_1$ and $\mathbf{X}_2$, the offspring matrix $\mathbf{X}_{\text{offspring}}$ is generated using operations such as addition, subtraction, or multiplication. Infeasible offsprings are repaired to meet constraints.

2) Mutation: To introduce diversity and explore new regions of the search space, mutation is performed on every offspring with a probability of p_m. The mutation involves two primary operations: modifying task sequences and altering task assignments. For the task sequence of an offspring, either a swap or an insertion mutation is performed. That is, two random tasks are exchanged or a random task is moved to a different position in the sequence.

Concurrently, for the task assignment matrix of the offspring, a random integer $a \in [1, M]$ is generated first as the flipping times. In each flipping operator, a random integer $b \in [1, N]$ is generated first, and then a UAV and b tasks are randomly selected to flip their assignment status, i.e., $x_{ij} \leftarrow 1 - x_{ij}$ for the selected UAV u_i and every selected task t_j.

3) Repair Mechanism: After the crossover and mutation, new offsprings are repaired to ensure that all constraints are satisfied. For each task t_j, if it is not yet assigned to any UAV, UAVs that are not assigned to t_j will be randomly selected and assigned to t_j until their combined capability exceeds the growth rate γ_j of the task demand. That is, the combined capability of UAVs assigned to task t_j must satisfy the constraint $\sum_{u_i \in U_j} \mu_i \geq \gamma_j$, where U_j is the set of UAVs assigned to task t_j. This ensures that the outfire demand at each task location is met over time.

C. Local Search

To refine solutions and improve convergence, a local search is adopted to adjust the task sequence and the task assignment matrix.

1) Task Sequence: The local search explores neighboring solutions by making small adjustments to the task sequence $\mathbf{R}$. Two tasks in the sequence are randomly selected for swapping. A subsequence is randomly selected to reverse the task order. This helps to find a better task sequence for reducing the total travel distance of all UAVs.

2) Task Assignment: For the assignment matrix $\mathbf{X}$, a task assigned to a UAV is reassigned to another one by considering the capacities and capabilities of UAVs. In addition, two tasks assigned to two distant UAVs are swapped. This fine-tuning can balance the workload among UAVs to reduce the makespan.

At the end of each iteration, all new solutions are combined with the population and the non-dominated ones are reserved based on the crowding distance sorting to maintain diversity.

D. The Complete Algorithm

At the beginning, an initial population $\mathcal{P}$ of a size N_p is randomly generated. Infeasible individuals are repaired via the repair mechanism. All individuals are evaluated and the nondominated solutions are stored in an extra Pareto archive $\mathcal{A}$. In each generation, individuals are randomly selected from the population as parents. Crossover is performed on parents with a probability of p_c and mutation is applied with a probability of p_m to generate offsprings. After that, the repair mechanism is performed on the offsprings to meet constraints. Later, local search is applied to the offsprings. At the end of each generation, all offsprings are combined with the population and the archived solutions for nondominated sorting. The nondominated ones are used to update the archive $\mathcal{A}$. The population is updated by selecting individuals from the combined set of parents and offspring according to the nondominated sorting and crowding distance sorting. This procedure repeats until reaching the maximum number of generations $G_{\max}$. Finally, the archive $\mathcal{A}$ is taken as the algorithm output.

IV. Experiment

A. Test Instances

To evaluate the performance of DEGA, the recent instances in [9] are adopted for test. There are 16 small-scale, 8 medium-scale, and 4 large-scale test instances.

TABLE I: Hypervolume results of all algorithms

Instance	DEGA	ACACO	ILS	MSGA	CACS	DQ-EMO
$L\_20\_60\_1.51$	**3.02E+12**	6.70E+11(+)	2.78E+12(+)	2.95E+12(+)	2.96E+12(+)	2.89E+12(+)
$L\_80\_15\_0.76$	4.68E+12	1.19E+12(+)	4.57E+12(+)	**4.71E+12**(−)	4.71E+12(−)	4.70E+12(−)
$L\_80\_20\_0.7$	2.78E+12	9.11E+11(+)	2.69E+12(+)	**2.91E+12**(−)	2.90E+12(−)	2.85E+12(−)
$L\_80\_20\_1.12$	**1.88E+14**	5.04E+13(+)	1.82E+14(+)	1.87E+14(+)	1.87E+14(+)	1.87E+14(+)
$M\_15\_30\_2.16$	**2.59E+14**	1.50E+14(+)	2.12E+14(+)	2.51E+14(+)	2.50E+14(+)	2.45E+14(+)
$M\_20\_20\_0.58$	3.74E+08	1.16E+08(+)	3.57E+08(+)	3.87E+08(−)	**3.88E+08**(−)	3.76E+08(−)
$M\_20\_20\_0.967$	1.11E+13	3.72E+12(+)	1.02E+13(+)	1.14E+13(−)	**1.14E+13**(−)	1.09E+13(+)
$M\_20\_20\_0.97$	**5.20E+09**	1.42E+09(+)	5.02E+09(+)	5.09E+09(+)	5.09E+09(+)	5.12E+09(+)
$M\_30\_30\_1.04$	**3.10E+11**	8.38E+10(+)	2.83E+11(+)	3.06E+11(+)	3.06E+11(+)	3.02E+11(+)
$M\_30\_30\_1.94$	**2.09E+16**	7.43E+15(+)	1.89E+16(+)	2.06E+16(+)	2.06E+16(+)	2.04E+16(+)
$M\_40\_10\_0.67$	7.03E+10	2.43E+10(+)	6.46E+10(+)	7.41E+10(−)	**7.44E+10**(−)	7.25E+10(−)
$M\_40\_15\_0.67$	1.46E+10	4.01E+09(+)	1.36E+10(+)	1.57E+10(−)	**1.57E+10**(−)	1.52E+10(−)
$S\_10\_10\_3.79$	**3.47E+17**	2.78E+17(+)	2.85E+17(+)	2.98E+17(+)	3.20E+17(+)	2.83E+17(+)
$S\_10\_15\_1.3$	1.17E+10	3.96E+09(+)	9.12E+09(+)	1.15E+10(+)	**1.17E+10**(≈)	1.05E+10(+)
$S\_10\_5\_1.39$	**5.63E+10**	3.47E+10(+)	4.42E+10(+)	5.41E+10(+)	5.51E+10(+)	4.87E+10(+)
$S\_15\_10\_1.17$	**1.21E+11**	2.66E+10(+)	1.06E+11(+)	1.18E+11(+)	1.19E+11(+)	1.18E+11(+)
$S\_15\_20\_0.67$	1.91E+08	6.34E+07(+)	1.63E+08(+)	1.95E+08(−)	**1.97E+08**(−)	1.80E+08(+)
$S\_17\_23\_1.71$	**3.54E+11**	2.04E+11(+)	3.28E+11(+)	3.37E+11(+)	3.35E+11(+)	3.35E+11(+)
$S\_20\_10\_0.47$	1.40E+08	3.71E+07(+)	1.31E+08(+)	**1.41E+08**(−)	1.41E+08(−)	1.39E+08(+)
$S\_20\_10\_0.94$	**6.68E+09**	1.44E+09(+)	6.12E+09(+)	6.66E+09(+)	6.68E+09(≈)	6.63E+09(+)
$S\_30\_10\_0.65$	2.02E+10	5.23E+09(+)	1.95E+10(+)	2.04E+10(−)	**2.05E+10**(−)	2.04E+10(−)
$S\_30\_10\_1.34$	**3.92E+12**	7.52E+11(+)	3.52E+12(+)	3.81E+12(+)	3.85E+12(+)	3.78E+12(+)
$S\_3\_10\_1.51$	9.65E+06	4.78E+06(+)	8.20E+06(+)	9.16E+06(+)	**9.71E+06**(≈)	8.49E+06(+)
$S\_3\_15\_5.03$	**4.09E+12**	3.49E+12(+)	3.26E+12(+)	2.78E+12(+)	3.54E+12(+)	2.13E+12(+)
$S\_5\_10\_0.93$	**1.78E+07**	3.89E+06(+)	1.55E+07(+)	1.66E+07(+)	1.77E+07(≈)	1.60E+07(+)
$S\_5\_10\_3.67$	**3.28E+13**	2.70E+13(+)	2.78E+13(+)	2.69E+13(+)	3.11E+13(+)	2.50E+13(+)
$S\_5\_40\_3.95$	**1.10E+10**	9.38E+09(+)	7.05E+09(+)	9.10E+09(+)	9.54E+09(+)	8.41E+09(+)
$S\_5\_4\_0.39$	3.48E+05	2.07E+04(+)	2.49E+05(+)	**3.56E+05**(≈)	3.31E+05(+)	2.68E+05(+)
Total		28 / 0 / 0	28 / 0 / 0	18 / 1 / 9	15 / 4 / 9	22 / 0 / 6
Best	16	0	0	4	8	0

The number of UAVs ranges from 3 to 80, and the number of tasks ranges from 4 to 60. Each instance is identified by a specific notation, such as $L\_20\_60\_1.51$, which represents a scenario with 20 UAVs and 60 tasks. The last number in the notation, 1.51, indicates the ratio of total task demand growth rate to total UAV capacity, which determines the difficulties of the scheduling problem relative to available UAV resources. In each instance, UAVs must coordinate to complete a set of distributed tasks efficiently. The solving difficulties of instances increase as the number of tasks grows, or as the ratio of task demand to UAV capacity rises.

B. Competing Algorithms

Five representative algorithms were selected for comparison: iterated local search (ILS) [10], adaptive coordination ant colony optimization (ACACO) [9], multi-strategy genetic algorithm (MSGA) [8], cooperative ant colony system (CACS) [7], and deep Q-learning with evolutionary many-objective optimization (DQ-EMO) [11].

The hypervolume metric was used to evaluate solution quality and diversity, with the reference point set at 1.1 times the maximum objective values found by all algorithms. A larger hypervolume indicates better performance.

For fair comparison, all algorithms used the same computational budget. DEGA parameters were set as follows: population size $N_p = M \times N$, crossover probability $P_c = 0.8$, mutation probability $P_m = 0.05$, maximum generations $G_{max} = 100$, and local search evaluation budget $K_{max} = 0.1 \times N_p$. The total function evaluation budget was $E = 110 \times M \times N$ for all algorithms. Parameters for competing algorithms followed their original references.

Each algorithm was executed 20 times independently on each test instance. Statistical significance was assessed using the Wilcoxon rank-sum test at a 0.05 significance level, with results denoted as "+" (DEGA significantly better), "≈" (DEGA approximately equal), and "−" (DEGA significantly worse).

C. Experimental Results

The hypervolume results are reported in Table I. The results demonstrate that DEGA outperforms both ACACO

TABLE II: Hypervolume results of DEGA and its variants

Test Cases	DEGA	noLS	noCrossover	noMutation
L_20_60_1.51	2.19E+09	**2.20E+09**(≈)	8.24E+08(+)	2.20E+09(≈)
L_80_15_0.76	**6.40E+09**	6.34E+09(+)	2.22E+09(+)	6.34E+09(+)
L_80_20_0.7	**1.06E+11**	1.06E+11(≈)	4.53E+10(+)	1.05E+11(+)
L_80_20_1.12	**3.52E+14**	3.50E+14(+)	1.37E+14(+)	3.50E+14(+)
Rank Sum Test	–	2 / 2 / 0	4 / 0 / 0	3 / 1 / 0

and ILS on all 28 instances. The poor performance of ACACO and ILS can be attributed to their inefficient search strategies, which easily violate the complex constraints of path planning and task assignment. In addition, DEGA performs better than MSGA, CACS, and DQ-EMO in 18, 15, and 22 out of 28 instances, while worse in 9, 9, and 6 instances, respectively. Specially, MSGA performs better if the number of UAVs significantly exceeds the number of tasks, achieving the best hypervolume in 4 instances. CACS demonstrates an effective ability to adapt flight routes and performs well in some mid-scale instances, achieving the best hypervolume in 8 small- or mid-scale cases. Nevertheless, DEGA exhibits superior performance in most instances and performs the best on 16 out of 28 instances, followed by CACS with 8 instances only. In addition, DEGA performs the best on almost on complex instances with less UAVs but more tasks. These instances are more difficult to plan the task execution sequence of UAVs. DEGA performs well on such instances due to the dual-encoding scheme, which enables to search feasible solutions efficiently. In summary, DEGA demonstrates good performance on the bi-objective scheduling problem.

D. Ablation Study

To further investigate the contribution of each component in DEGA, we conducted an ablation study by testing three variants of DEGA: DEGA without local search (noLS), DEGA without crossover (noCrossover), and DEGA without mutation (noMutation). Particularly, noLS removes local search, noCrossover does not use the crossover operator, and noMutation omits the mutation operator.

As shown in Table II, DEGA significantly outperforms noLS, noCrossover, and noMutation on 2, 4, and 3 out of 4 instances, whereas worse on 0, 0, and 0 instance, respectively. This shows that the removal of any component will lead to performance degradation. Particularly, crossover and mutation helps to generate diverse solution so as to explore the search space, and local search helps to improve convergence. Therefore, all three components are essential to enhance the performance of DEGA.

E. Parameter Studies

To investigate parameter sensitivity, experiments were conducted on 16 small-scale instances with different values of crossover probability (P_c), mutation probability (P_m), and local search evaluation budget (K_{max}). Results showed that $P_c = 0.8$, $P_m = 0.05$, and $K_{max} = 0.1 \times N_p$ generally yielded the best performance. Statistical tests revealed no significant differences between parameter values, indicating DEGA's robustness to parameter settings. These parameters were therefore adopted for all experiments in this study.

V. Conclusion

This paper develops a dual-encoding-based genetic algorithm (DEGA) to solve the multi-objective multi-UAV scheduling problem. DEGA leverages a dual-encoding scheme to plan the task execution order and task assignment so as to handle the complex constraints. Crossover and mutation operators are designed based on the encoding scheme to improve solution diversity. Local search is also adopted to further improve solution quality. Experimental results show that DEGA performs better than state-of-the-art algorithms on most instances especially with limited UAV resources in terms of solution diversity and optimality. In addition, Each component of DEGA is important to improve algorithm performance.

References

[1] K. Telli, O. Kraa, Y. Himeur, A. Ouamane, M. Boumehraz, S. Atalla, and W. Mansoor, "A comprehensive review of recent research trends on unmanned aerial vehicles (uavs)," *Systems*, vol. 11, no. 8, p. 400, 2023.

[2] X. Li, J. Tupayachi, A. Sharmin, and M. Martinez Ferguson, "Drone-aided delivery methods, challenges, and the future: A methodological review," *Drones*, vol. 7, no. 3, p. 191, 2023.

[3] B. Y. Lattimer, X. Huang, M. A. Delichatsios, Y. A. Levendis, K. Kochersberger, S. Manzello, P. Frank, T. Jones, J. Salvador, and C. Delgado, "Use of unmanned aerial systems in outdoor firefighting," *Fire Technology*, vol. 59, no. 6, pp. 2961–2988, 2023.

[4] E. Seraj, A. Silva, and M. Gombolay, "Multi-uav planning for cooperative wildfire coverage and tracking with quality-of-service guarantees," *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 2, p. 39, 2022.

[5] Y. Liu, X. Li, J. Wang, F. Wei, and J. Yang, "Reinforcement-learning-based multi-uav cooperative search for moving targets in 3d scenarios," *Drones*, vol. 8, no. 8, p. 378, 2024.

[6] A. Cardon, T. Galinho, and J.-P. Vacher, "Genetic algorithms using multi-objectives in a multi-agent system," *Robotics and Autonomous Systems*, vol. 33, no. 2-3, pp. 179–190, 2000.

[7] T. Qian, X.-F. Liu, and Y. Fang, "A cooperative ant colony system for multi-objective multi-robot task allocation with precedence constraints," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2024.

[8] Y. Shen and H. Li, "A multi-strategy genetic algorithm for solving multi-point dynamic aggregation problems with priority relationships of tasks," *Electronic Research Archive*, vol. 32, no. 1, pp. 445–472, 2024.

[9] G. Gao, Y. Mei, Y.-H. Jia, W. N. Browne, and B. Xin, "Adaptive coordination ant colony optimization for multi-point dynamic aggregation," *IEEE Transactions on Cybernetics*, vol. 52, no. 8, pp. 7362–7376, 2021.

[10] X. Wang, T.-M. Choi, Z. Li, and S. Shao, "An effective local search algorithm for the multi-depot cumulative capacitated vehicle routing problem," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 12, pp. 4948–4958, 2019.

[11] M. Li, Z. Wang, K. Li, X. Liao, K. Hone, and X. Liu, "Task Allocation on Layered Multiagent Systems: When Evolutionary Many-Objective Optimization Meets Deep Q-Learning," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 842–855, Oct. 2021.